

> Parking_data

Distance	Time of Day	Vehicle Density	Parking Duration	Status
50	9	20	30	Occupied
120	14	60	90	Occupied
80	11	45	60	Occupied
30	8	15	20	Vacant
800	18	80	120	Occupied
60	10	30	45	Vacant
150	16	70	100	Occupied
40	7	10	15	Vacant
100	13	85	70	Occupied
170	19	75	110	Occupied

Ex: No: 5A
Date: 24/3/26
Smart Parking Slot Occupancy Classification Using K-Nearest Neighbor Algorithm

Problem Description:

In a smart city environment, parking management systems collect sensor data to determine whether parking slots are **Occupied** or **Vacant**. Each parking record includes numerical attributes such as **Distance from entry gate**, **Time of day**, **Vehicle density**, and **Parking duration**.

The objective is to classify parking slot status using the **K-Nearest Neighbor (KNN)** algorithm and visualize the classification results for efficient parking management.

Objective:

To implement and visualize the K-Nearest Neighbor algorithm in R for classifying parking slot occupancy based on sensor data.

Procedure:

- Step 1: Create Parking Dataset

	Distance	Time of Day	Vehicle Density	Parking Duration	Status
1	50	9	20	30	Occupied
2	120	14	60	90	Occupied
3	80	11	45	60	Occupied
4	30	8	15	20	Vacant
5	200	18	80	120	Occupied
6	60	10	30	45	Vacant
7	150	16	70	100	Occupied
8	40	7	10	15	Vacant
9	100	13	85	70	Occupied
10	170	19	75	110	Occupied

Parking_data <- data.frame (

Distance = c(50, 120, 80, 30, 200, 60, 150, 40, 100, 170),

Time of Day = c(9, 14, 11, 8, 18, 10, 16, 7, 13, 19),

Vehicle Density = c(20, 60, 45, 15, 80, 30, 70, 10, 85, 75),

Parking Duration = c(30, 90, 60, 20, 120, 45, 100, 15, 70, 110),

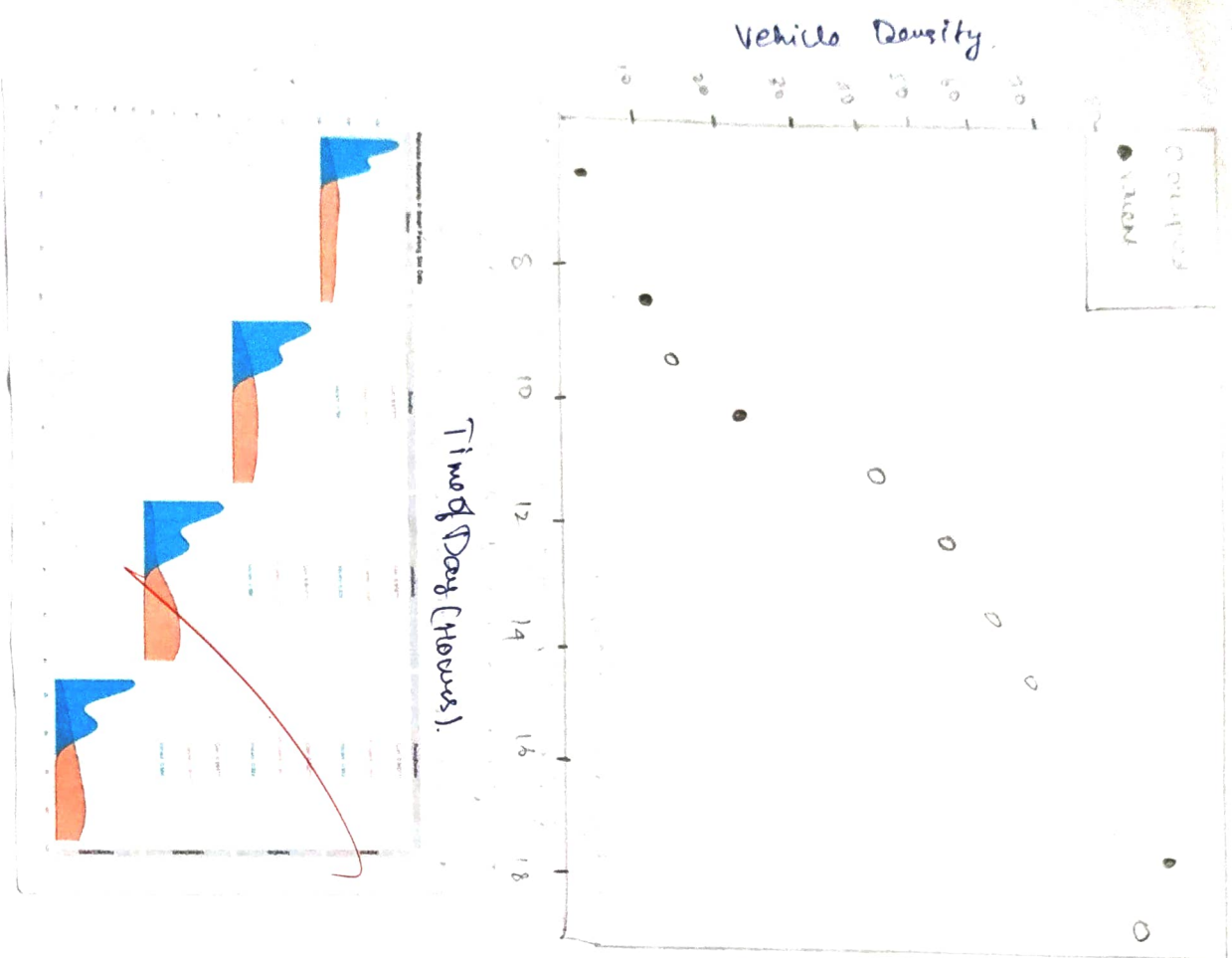
Status = c("Occupied", "Occupied", "Occupied", "Vacant",

"Occupied", "Vacant", "Occupied", "Vacant", "Occupied",

"Occupied")

)
Parking_data

Parking slot status Distribution.



- Step 2: Select Features and Class Label

features $\leftarrow$ parking_data[:, 2:4]

labels $\leftarrow$ as_data (parking_data['status'])

- Step 3: Data Visualization (Scatter Plot) - TimeOfDay and VehicleDensity.

Plot (features of Time of Day, features of Vehicle Density,

col = 08. numeric (labels),

Pch = 19,

mlab = "Time of Day (Hours)",

ylab = "Vehicle Density",

main = "Parking slot status Distribution")

legend ("topleft",

legend = c("Completed",

col = 1:2,

Pch = 19)

- Step 4: Normalize the Data

Normalize $\leftarrow$ function (x) {

$(x - \min(x)) / (\max(x) - \min(x))$

}

features_norm $\leftarrow$ as_data.frame(apply(features, nrow(features),

> Predicted - labels

C1) Occupied Vacant Occupied
 levels: Occupied Vacant.
 > confusion_matrix.

Actual	Predicted	
	occupied	Vacant
occupied	2	0
Vacant	0	1.

> accuracy.

1.

- Step 5: Train(70%) - Test Split(30%)

$index \leftarrow sample(1:nrow(features_norm), 0.7, norm(features_norm))$
 $train_data \leftarrow features_norm[index,]$
 $test_data \leftarrow features_norm[-index,]$
 $train_labels \leftarrow labels[index]$
 $test_labels \leftarrow labels[-index].$

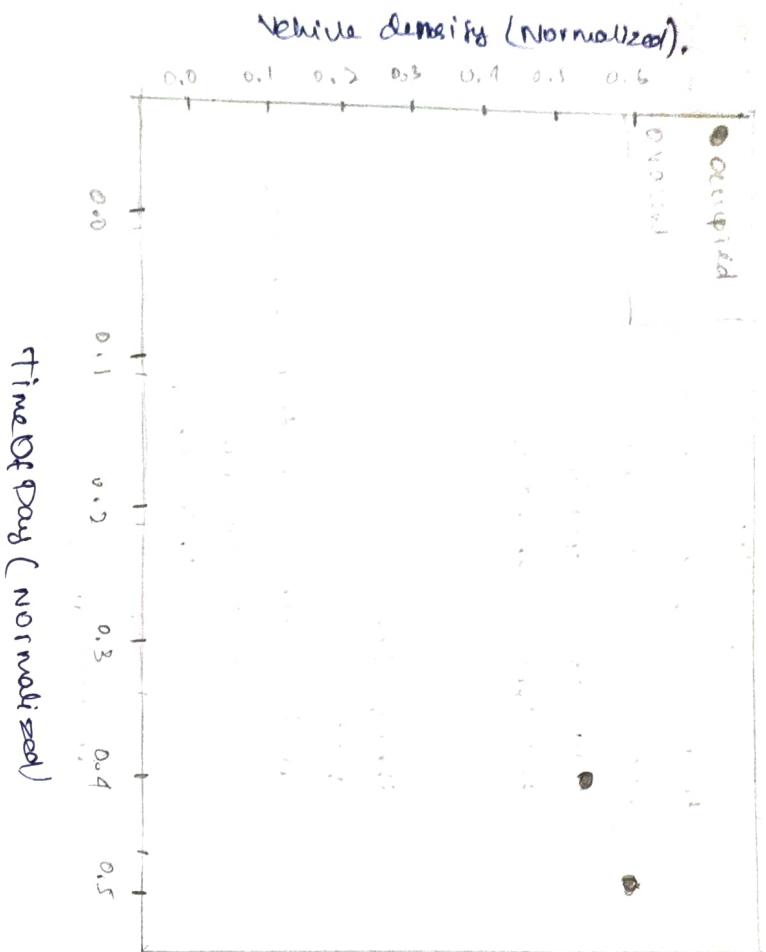
- Step 6: Apply KNN Algorithm - $k=3$ and predict labels

$library(kknn)$
 $k \leftarrow 3$
 $predicted_labels \leftarrow kknn(k, train = train_data,$
 $test = test_data,$
 $cl = train_labels,$
 $k = k)$
 $predicted_labels.$

- Step 7: Confusion Matrix and Accuracy

$confusion_matrix \leftarrow table($
 $actual = test_labels,$
 $predicted = predicted_labels)$
 $confusion_matrix,$
 $accuracy \leftarrow sum(diag(confusion_matrix)) / sum(confusion_matrix)$
 $accuracy$

kNN Classification Result for parking slot.



- Step 8: Visualization of Classification Result - Normalized data - TimeOfDay and VehicleDensity,

```
plot (test_data $ timeOfDay, test_data $ vehicle_Density,
      col = as.numeric (predicted_labels),
      pch = "a",
      xlab = "Time of Day (Normalized)",
      ylab = "Vehicle Density (Normalized)",
      main = "kNN Classification Result for parking slots",
      legend ("topleft", legend = levels (predicted_labels),
              col = 1:2, pch = "a")
```

- Step 8: Pairwise Visualization using ggpairs: To visualize relationships between multiple numerical attributes and observe how they vary with parking slot status.

```
install.packages ("Ggally")
library (Ggally)
ggpairs (parking_data,
         columns = 1:4
         aes (color = status, alpha = 0.7),
         title = "pairwise Relationship in Smart Parking
               Slot Data")
```

Result:

Thus, the program successfully demonstrates the use of the K-Nearest Neighbor algorithm in R to classify smart parking slot occupancy. The visualizations effectively represent parking patterns, and the model provides accurate classification suitable for smart city traffic and parking management systems.

Ex:No: 5B

Date: 20/3/22

Visualization and Classification of Iris Flower Species Using K-Nearest Neighbor Algorithm

Problem Description:

The Iris dataset is a well-known dataset in machine learning that contains measurements of iris flowers belonging to three different species: *Setosa*, *Versicolor*, and *Virginica*. Each flower is described using four numerical attributes: **Sepal Length**, **Sepal Width**, **Petal Length**, and **Petal Width**.

The objective is to visualize the dataset, apply the K-Nearest Neighbor (KNN) algorithm, and analyze the classification results.

Objective:

To visualize the Iris dataset and implement the K-Nearest Neighbor algorithm in R for classifying iris flower species based on their measurements.

Procedure:

- **Step 1: Install and Load Required Libraries**

We install and load the necessary libraries for implementing the K-NN algorithm.

```
# Install necessary libraries
```

```
install.packages("dplyr")  
install.packages("caret")  
install.packages("ggplot2")  
install.packages("knn")
```

```
# Load the libraries
```

```
library(dplyr)  
library(caret)  
library(ggplot2)  
library(knn)
```

- **Step 2: Load and Prepare the Dataset**

We use the iris dataset for classification and check its structure.

```
data(iris)  
set.seed(123)
```

Normalize the numerical data (Feature Scaling)

```
normalize <- function(x) {  
  return ((x - min(x)) / (max(x) - min(x)))  
}
```

Apply normalization to numerical columns

```
iris_norm <- as.data.frame(lapply(iris[1:4],  
normalize))
```

```
iris_norm$species <- iris$species
```

- **Step 3: Split the Dataset into Training and Testing Sets**

We split the dataset into 80% training and 20% testing.

```
split_index <- createDataPartition(iris_norm$species, p=0.8,  
list=FALSE)
```

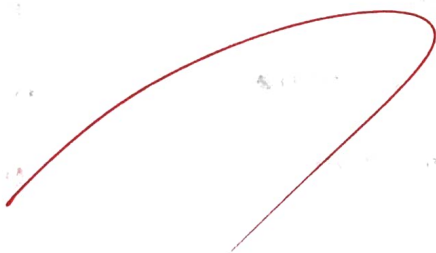
```
train_data <- iris_norm[split_index]
```

```
test_data <- iris_norm[-split_index,]
```

Extract labels (target variable)

```
train_labels <- train$species
```

```
test_labels <- test_data$species
```



Print (conf-matrix)

confusion matrix and statistics

	reference		
prediction	setosa	versicol	virginica
setosa	10	0	0
versicol	0	10	0
virginica	0	0	8

Overall statistics

Recalling : 0.9333

q57.ct : (0.7793, 0.918)

No Information Rate : 0.3333

P-value [acc > NLR] : 8.747e-12

Kappa : 0.9

McNemar's test P-value : NA

Statistics by class:

	class: setosa	class: versicol	class: virginica
Sensitivity	1.0000	1.0000	0.8000
Specificity	1.0000	0.9000	1.0000
Pos Pred value	1.0000	0.9333	1.0000
Neg Pred value	1.0000	1.0000	0.9091
Prevalence	0.3333	0.3333	0.3333
Detection Rate	0.3333	0.3333	0.2667
Detection Prevalence	0.3333	0.4000	0.2667
Balanced Accuracy	1.0000	0.9500	0.9000

Step 4: Implement the K-Nearest Neighbor Algorithm

We use knn() function to build the model.

k <- 5 # Choose number of neighbors

knn.predictions <- knn(train = train_data[, 1:4],

test = test_data[, 1:4])

cl = train_data[, k:k]

Step 5: Evaluate Model Performance

We compute accuracy using a confusion matrix.

conf-matrix <- confusionMatrix(knn.predictions, test_labels,
print(conf-matrix))

Step 6: Visualize the Classification Results

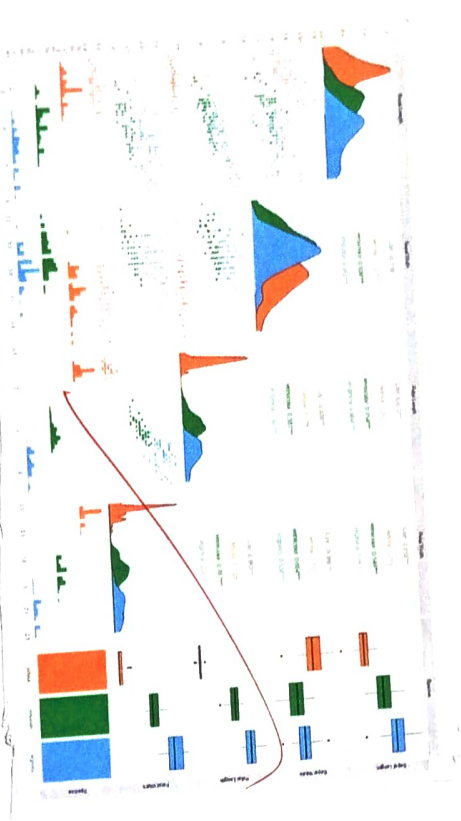
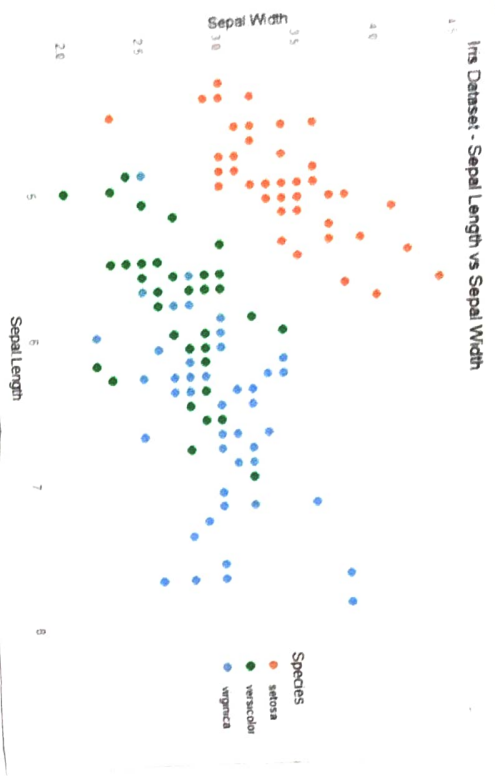
We plot a scatter plot of Sepal.Length vs Sepal.Width with predicted class labels.

Scatter Plot: Sepal.Length vs Sepal.Width, colored by Species

ggplot(trim, aes (x = Sepal.Length, y = Sepal.Width, color =

Species)) + geom_point(size = 3) +

labs(title = "Iris Data - Sepal.Length vs Sepal.Width",
width = "Sepal.Length", y = "Sepal.Width") + theme_minimal()



Now, we generate a pair plot using the `ggpairs()` function, which visualizes the relationships between all numeric features.

Pair Plot: Relationship between all numeric features

`ggpairs (iris, aes (color = species))`

Result:

Thus, the program successfully demonstrates the visualization and classification of the Iris dataset using the K-Nearest Neighbor algorithm in R. The visualizations reveal clear class separations, and the KNN model achieves high classification accuracy, confirming its effectiveness for multi-class classification problems.